Lab Work - 15th August

Name: Arnav Singhal
MIS: 612203012
SY B. Tech. Comp. Div 1
Batch: S1

Conditional Statements

1. Write a C program to find maximum between two numbers.

Code:

```
#include <stdio.h>

int main() {
	int a, b;
	printf("Enter the first number: ");
	scanf("%d", &a);
	printf("Enter the second number: ");
	scanf("%d", &b);

	if(a > b)
		printf("Maximum = %d\n", a);
	else
		printf("Maximum = %d\n", b);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q1.c -o Q1
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q1
Enter the first number: 25
Enter the second number: 100
Maximum = 100
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

2. Write a C program to check whether a number is negative, positive or zero.

Code:

```
#include <stdio.h>

int main() {
	int num;
	printf("Enter a number: ");
	scanf("%d", &num);

	if(num < 0)
		printf("The number is negative\n");
	else if(num > 0)
		printf("The number is positive\n");
	else
		printf("The number is zero\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q2.c -o Q2
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q2
Enter a number: -50
The number is negative
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

3. Write a C program to check whether a number is divisible by 5 and 11 or not.

Code:

```
#include <stdio.h>

int main() {
	int num;
	printf("Enter a number: ");
	scanf("%d", &num);
```

```
        if(num % 5 == 0 && num % 11 == 0)
                printf("%d is divisible by 5 and 11\n", num);
        else if(num % 5 == 0)
                printf("%d is divisible by 5 but not by 11\n", num);
        else if(num % 11 == 0)
                printf("%d is divisible by 11 but not by 5\n", num);
        else
                printf("%d is divisible by neither 5 nor 11\n", num);

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q3.c -o Q3
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q3
Enter a number: 40
40 is divisible by 5 but not by 11
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

4. Write a C program to check whether a number is even or odd.

Code:

```
#include <stdio.h>

int main() {
        int num;
        printf("Enter a number: ");
        scanf("%d", &num);

        if(num % 2 == 0)
                printf("%d is an even number\n", num);
        else
                printf("%d is an odd number\n", num);

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q4.c -o Q4
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q4
Enter a number: 50
50 is an even number
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

5. Write a C program to check whether a year is leap year or not.

Code:

```
#include <stdio.h>

int main() {
	int year;
	printf("Enter a year: ");
	scanf("%d", &year);

	if(year % 100 == 0) {
		if(year % 400 == 0)
			printf("%d is a leap year\n", year);
		else
			printf("%d is not a leap year\n", year);
	}
	else if(year % 4 == 0)
		printf("%d is a leap year\n", year);
	else
		printf("%d is not a leap year\n", year);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q5.c -o Q5
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q5
Enter a year: 2100
2100 is not a leap year
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

6. Write a C program to check whether a character is alphabet or not.

Code:

```
#include <stdio.h>

int main() {
	char c;
	printf("Enter a character: ");
	scanf("%c", &c);

	if((c >= 65 && c <= 90) || (c >= 97 && c <= 122))
		printf("The character is an alphabet\n");
	else
		printf("The character is not an alphabet\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q6.c -o Q6
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q6
Enter a character: h
The character is an alphabet
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

7. Write a C program to input any alphabet and check whether it is vowel or consonant.

Code:

```
#include <stdio.h>

int main() {
	char c;
	printf("Enter an alphabet: ");
	scanf("%c", &c);

	if(c >= 97 && c <= 122)
		c -= 32;

	if(c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U')
		printf("The entered alphabet is a vowel\n");
```

```
        else
                printf("The entered alphabet is a consonant\n");

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q7.c -o Q7
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q7
Enter an alphabet: a
The entered alphabet is a vowel
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

8. Write a C program to input any character and check whether it is alphabet, digit or special character.

Code:

```
#include <stdio.h>

int main() {
        char ch;
        printf("Enter a character: ");
        scanf("%c", &ch);

        if((ch >= 65 && ch <= 90) || (ch >= 97 && ch <= 122))
                printf("The entered character is an alphabet\n");
        else if(ch >= 48 && ch <= 57)
                printf("The entered character is a digit\n");
        else
                printf("The entered character is a special character\n");

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q8.c -o Q8
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q8
Enter a character: l
The entered character is an alphabet
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

9. Write a C program to check whether a character is uppercase or lowercase alphabet.

Code:

```
#include <stdio.h>

int main() {
	char ch;
	printf("Enter a character: ");
	scanf("%c", &ch);

	if(ch >= 65 && ch <= 90)
		printf("The entered character is an uppercase alphabet\n");
	else if(ch >= 97 && ch <= 122)
		printf("The entered character is a lowercase alphabet\n");
	else
		printf("The entered character is not an alphabet\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q9.c -o Q9
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q9
Enter a character: P
The entered character is an uppercase alphabet
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

10. Write a C program to input month number and print number of days in that month.

Code:

```
#include <stdio.h>

int main() {
	int month_num;
	printf("Enter the month number: ");
	scanf("%d", &month_num);
	if(month_num <= 7) {
		if(month_num == 2)
			printf("%d", 28);
		else if(month_num % 2)
			printf("%d", 31);
```

```
            else
                printf("%d", 30);
        }
        else {
            if(month_num % 2 == 0)
                printf("%d", 31);
            else
                printf("%d", 30);
        }
        printf("\n");
        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q10.c -o Q10
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q10
Enter the month number: 9
30
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

11. Write a C program to input angles of a triangle and check whether triangle is valid or not.

Code:

```
#include <stdio.h>

int main() {
        int a, b, c;
        printf("Enter the angles of the triangle:\n");
        scanf("%d", &a);
        scanf("%d", &b);
        scanf("%d", &c);

        if((a + b + c) == 180)
            printf("The triangle is valid\n");
        else
            printf("The triangle is invalid\n");

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q11.c -o Q11
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q11
Enter the angles of the triangle:
25
70
85
The triangle is valid
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

12. Write a C program to check whether the triangle is equilateral, isosceles or scalene triangle.

Code:

```
#include <stdio.h>

int main() {
	int a, b, c;
	printf("Enter the length of the first side: ");
	scanf("%d", &a);
	printf("Enter the length of the second side: ");
	scanf("%d", &b);
	printf("Enter the length of the third side: ");
	scanf("%d", &c);

	if(a == b && b == c)
		printf("Equilateral triangle\n");
	else if(a == b || b == c || c == a)
		printf("Isosceles triangle\n");
	else
		printf("Scalene triangle\n");
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q12.c -o Q12
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q12
Enter the length of the first side: 25
Enter the length of the second side: 20
Enter the length of the third side: 15
Scalene triangle
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

13. Write a C program to find all roots of a quadratic equation.

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
	float a, b, c, d, x1, x2;
	printf("Enter the coefficient of x^2: ");
	scanf("%f", &a);
	printf("Enter the coefficient of x: ");
	scanf("%f", &b);
	printf("Enter the constant term: ");
	scanf("%f", &c);

	d = b * b - (4 * a * c);

	if(d >= 0) {
		printf("x = %f, %f\n", ((-b + sqrt(d))/ (2 * a)), ((-b - sqrt(d))/ (2 * a)));
	}
	else{
		printf("Roots are imaginary as discriminant is negative\n");
		x1 = -b / (2 * a);
		x2 = sqrt(-d) / (2 * a);
		printf("x = %f + %fi, %f - %fi\n", x1, x2, x1, x2);
	}

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q13.c -o Q13 -lm
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q13
Enter the coefficient of x^2: 5
Enter the coefficient of x: 3
Enter the constant term: 15
Roots are imaginary as discriminant is negative
x = -0.300000 + 1.705872i, -0.300000 - 1.705872i
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

14. Write a C program to input marks of five subjects Physics, Chemistry, Biology, Mathematics and Computer. Calculate percentage and grade according to following:
Percentage >= 90% : Grade A
Percentage >= 80% : Grade B
Percentage >= 70% : Grade C
Percentage >= 60% : Grade D
Percentage >= 40% : Grade E
Percentage < 40% : Grade F

Code:

```
#include <stdio.h>

int main() {
	float phy, chem, bio, math, comp, per;
	printf("Enter marks scored in Physics: ");
	scanf("%f", &phy);
	printf("Enter marks scored in Chemistry: ");
	scanf("%f", &chem);
	printf("Enter marks scored in Biology: ");
	scanf("%f", &bio);
	printf("Enter marks scored in Mathematics: ");
	scanf("%f", &math);
	printf("Enter marks scored in Computer: ");
	scanf("%f", &comp);

	per = (phy + chem + bio + math + comp) / 5;
	printf("Percentage = %f%%\n", per);
	printf("Grade = ");

	if(per >= 90)
		printf("A");
	else if(per >= 80)
		printf("B");
	else if(per >= 70)
		printf("C");
	else if(per >= 60)
		printf("D");
	else if(per >= 40)
		printf("E");
```

```
        else
                printf("F");
        printf("\n");

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q14.c -o Q14
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q14
Enter marks scored in Physics: 90
Enter marks scored in Chemistry: 70
Enter marks scored in Biology: 80
Enter marks scored in Mathematics: 85
Enter marks scored in Computer: 95
Percentage = 84.000000%
Grade = B
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

15. Write a C program to input basic salary of an employee and calculate its Gross salary according to following:
Basic Salary <= 10000 : HRA = 20%, DA = 80%
Basic Salary <= 20000 : HRA = 25%, DA = 90%
Basic Salary > 20000 : HRA = 30%, DA = 95%

Code:

```
#include <stdio.h>

int main() {
        int basic_sal, hra, da;

        printf("Enter the basic salary of the employee: ");
        scanf("%d", &basic_sal);

        if(basic_sal <= 10000) {
                hra = 20 * basic_sal / 100;
                da = 80 * basic_sal / 100;
        }
        else if(basic_sal <= 20000) {
                hra = 25 * basic_sal / 100;
                da = 90 * basic_sal / 100;
        }
```

```
        else {
                hra = 30 * basic_sal / 100;
                da = 95 * basic_sal / 100;
        }

        printf("Gross salary = %d\n", (basic_sal + hra + da));
        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q15.c -o Q15
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q15
Enter the basic salary of the employee: 80000
Gross salary = 180000
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

16. Write a C program to input electricity unit charges and calculate total electricity bill according to the given condition:
For first 50 units Rs. 0.50/unit
For next 100 units Rs. 0.75/unit
For next 100 units Rs. 1.20/unit
For unit above 250 Rs. 1.50/unit
An additional surcharge of 20% is added to the bill

Code:

```
#include <stdio.h>

int main() {
        int units;
        float cost, surcharge;

        printf("Enter number of units consumed: ");
        scanf("%d", &units);

        if(units <= 50)
                cost = units * 0.5;
        else if(units <= 150)
                cost = 50 * 0.5 + (units - 50) * 0.75;
        else if(units <= 250)
                cost = 50 * 0.5 + 100 * 0.75 + (units - 150) * 1.2;
```

```
        else
                cost = 50 * 0.5 + 100 * 0.75 + 150 * 1.2 + (units - 250) * 1.5;

        surcharge = 0.2 * cost;

        printf("Total electricity bill = Rs. %.2f\n", cost + surcharge);

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ gcc -Wall Q16.c -o Q16
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$ ./Q16
Enter number of units consumed: 200
Total electricity bill = Rs. 192.00
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Conditional Statements$
```

Control Statements

Display output of the following code:

```
#include "stdio.h"
int main()
{
	int i = 1, j;
	for ( ; ; )
	{
		if (i)
			j = --i;
		if (j < 10)
			printf("HelloWorld", j++);
		else
			break;
	}
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$ gcc output20.c -o output20
output20.c: In function ‘main’:
output20.c:11:20: warning: too many arguments for format [-Wformat-extra-args]
   11 |             printf("HelloWorld", j++);
      |                    ^~~~~~~~~~~~
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$
```

1. Write a program to read 10 integers. Display these numbers by printing three numbers in a line separated by commas.

Code:

```
#include <stdio.h>

int main() {
	int a[10], i;
	printf("Enter 10 integers:\n");
	for(i = 0; i < 10; i++)
		scanf("%d", &a[i]);
	for(i = 0; i < 9; i++) {
		printf("%d, ", a[i]);
```

```
            if(i % 3 == 2)
                printf("\n");
        }
        printf("%d\n", a[i]);
        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$ gcc -Wall numbers.c -o numbers
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$ ./numbers
Enter 10 integers:
1
2
3
4
5
6
7
8
9
10
1, 2, 3,
4, 5, 6,
7, 8, 9,
10
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Control Statements$
```

Arrays

1. Write a program that calculates the sum of squares of the elements of an integer array of size 10.

Code:

```
#include <stdio.h>

int main() {
	int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, sum = 0;
	for(int i = 0; i < 10; i++) {
		sum += (a[i] * a[i]);
	}
	printf("Sum of squares of the elements = %d\n", sum);
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$ gcc -Wall array_square_sum.c -o array_square_sum
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$ ./array_square_sum
Sum of squares of the elements = 385
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$
```

11. Reverse elements of array without using additional array. Eg input array – {10,45,3216,88} should change to {88,16,32,45,10}

Code:

```
#include <stdio.h>

int main() {
	int a[] = {10, 45, 3216, 88}, li, ui, temp;
	int n = sizeof(a)/sizeof(a[0]);
	li = 0;
	ui = n - 1;
	while(li < ui) {
		temp = a[li];
		a[li] = a[ui];
		a[ui] = temp;
```

```
            li++;
            ui--;
        }
        printf("{");
        for(int i = 0; i < n - 1; i++)
            printf("%d, ", a[i]);
        printf("%d}\n", a[n - 1]);
        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$ gcc -Wall reverse_array.c -o reverse_array
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$ ./reverse_array
{88, 3216, 45, 10}
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Array$
```

Strings

1. Write a function to count number of occurrences of a character in a string.

Code:

```
#include <stdio.h>

int count(char* str, char c) {
	int i = 0, occ = 0;
	while(str[i] != '\0') {
		if(str[i] == c)
			occ++;
		i++;
	}
	return occ;
}

int main() {
	char str[] = "data structures and algorithms", ch;
	printf("Enter the character for which the no of occurrences is to be found: ");
	scanf("%c", &ch);
	printf("Number of occurences of \'%c\' is %d\n", ch, count(str, ch));
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q1.c -o Q1
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q1
Enter the character for which the no of occurrences is to be found: d
Number of occurences of 'd' is 2
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

2. Write the strtok() function.

Code:

```
#include <stdio.h>
#include <string.h>

char* strtok(char* str, const char* delim) {
	static char* last = NULL;  // Static pointer to store the rest of the string between
calls
```

```
	if (str == NULL) {
		str = last;
	if (str == NULL)
		return NULL;
	}

	str += strspn(str, delim);

	if (*str == '\0') {
		last = NULL;
		return NULL;
	}

	char* end = str + strcspn(str, delim);

	if (*end != '\0') {
		*end = '\0';
		last = end + 1;
	} else {
		last = NULL;
	}

	return str;
}

int main() {
	char str[] = "data,structures,,and algorithms";
	char* token = strtok(str, ", ");
	while (token != NULL) {
		printf("%s\n", token);
		token = strtok(NULL, ", ");
	}
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q2.c -o Q2
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q2
data
structures
and
algorithms
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

3. Write a function which finds the longest possible subsequence of one string into another and returns the length + pointer to the subsequence.
For example, if the input is "abcdemnopxyz" and "mnotq" then "mno" is the longest subsequence of "mnotq" occuring in "abcdemnopxyz", so the function returns 3 and a pointer to "mno" inside "abcdemnopxyz"

Code:

```
#include <stdio.h>
#include <string.h>

int findLongestSubsequence(const char* str1, const char* str2, const char**
subsequence) {
	int len1 = strlen(str1);
	int len2 = strlen(str2);

	int start = 0;
	int maxLength = 0;

	for (int i = 0; i < len1; i++) {
		int currentLength = 0;
		int index1 = i;
		int index2 = 0;

		while (index1 < len1 && index2 < len2) {
			if (str1[index1] == str2[index2]) {
				currentLength++;
				index2++;
			}
			index1++;
		}

		if (currentLength > maxLength) {
```

```
                start = index2 + currentLength - 1;
                maxLength = currentLength;
            }
        }

        *subsequence = str1 + start;
        return maxLength;
}

int main() {
        const char* str1 = "abcdemnopxyz";
        const char* str2 = "mnotq";

        const char* subsequence;
        int length = findLongestSubsequence(str1, str2, &subsequence);

        printf("Longest subsequence: %.*s\n", length, subsequence);
        printf("Length of longest subsequence: %d\n", length);

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q3.c -o Q3
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q3
Longest subsequence: mno
Length of longest subsequence: 3
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

4. Write a function to find gcd() of 2 numbers. (See Dromey for this, after you have tried your bit)

Code:

```
#include <stdio.h>

int gcd(int num1, int num2) {
        if(num1 > num2) {
            if(num1 % num2 == 0)
                return num2;
            else
                return gcd(num1 % num2, num2);
```

```
        }
        else {
                if(num2 % num1 == 0)
                        return num1;
                else
                        return gcd(num1, num2 % num1);
        }
}

int main() {
        int n1, n2;
        printf("Enter the first number: ");
        scanf("%d", &n1);
        printf("Enter the second number: ");
        scanf("%d", &n2);

        printf("GCD of the numbers = %d\n", gcd(n1, n2));
        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q4.c -o Q4
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q4
Enter the first number: 50
Enter the second number: 5
GCD of the numbers = 5
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

5. Write a function to find lcm() of 2 numbers.

Code:

```
#include <stdio.h>

int lcm(int num1, int num2) {
        //LCM = Product of the numbers / GCD; Here the GCD is calculated first, and
then the LCM is computed from it
        int gcd, prod = num1 * num2;
        while(1) {
                if(num1 > num2) {
                        if(num1 % num2 == 0) {
                                gcd = num2;
```

```
                        break;
                }
                else
                        num1 = num1 % num2;
        }
        else {
                if(num2 % num1 == 0) {
                        gcd = num1;
                        break;
                }
                else
                        num2 = num2 % num1;
        }
    }

    return prod/gcd;
}

int main() {
    int n1, n2;
    printf("Enter the first number: ");
    scanf("%d", &n1);
    printf("Enter the second number: ");
    scanf("%d", &n2);

    printf("LCM of the numbers = %d\n", lcm(n1, n2));
    return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q5.c -o Q5
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q5
Enter the first number: 30
Enter the second number: 6
LCM of the numbers = 30
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

6. Write a function to convert a decimal number to a binary number and return the binary representation in a string.

Code:

```
#include <stdio.h>
#include <stdlib.h>

char* bin(int num) {
	if(num == 0)
		return "0";
	char* p = (char*)malloc(50 * sizeof(char));
	int i = 0, temp;
	while(num) {
		p[i+1] = p[i];
		p[i] = (num % 2)? '1':'0';
		num /= 2;
		i++;
	}

	//Reversing the string
	for(int j = 0; j < i/2; j++) {
		temp = p[j];
		p[j] = p[i - j - 1];
		p[i - j - 1] = temp;
	}

	p[i] = '\0';
	return p;
}

int main() {
	int num;
	printf("Enter a number: ");
	scanf("%d", &num);
	char* p = bin(num);
	printf("Binary representation: %s\n", p);
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q6.c -o Q6
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q6
Enter a number: 20
Binary representation: 10100
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

7. Write your own code for following library functions: strcasecmp strsep strcasecmp strcoll

Code:

```
#include <stdio.h>
#include <limits.h>

int strcasecmp(char* str1, char* str2) {
	int i = 0, diff;
	char a, b;
	while(str1[i] != '\0' && str2[i] != '\0') {
		a = str1[i];
		b = str2[i];

		if((a >= 65 && a <= 90) && (b >= 97 && b <= 122))
			diff = a - b + 32;
		else if((a >= 97 && a <= 122) && (b >= 65 && b <= 90))
			diff = a - b - 32;
		else
			diff = a - b;

		if(diff > 0)
			return INT_MAX;
		else if(diff < 0)
			return INT_MIN;

		i++;
	}

	if(str1[i] == '\0' && str2[i] == '\0')
		return 0;
	else if(str1[i] == '\0')
		return INT_MIN;
	else
```

```
		return INT_MAX;
}

char* strsep(char** stringp, char* delim) {
	if (*stringp == NULL) {
		return NULL;
	}

	char* start = *stringp;
	char* p = start;

	while (*p != '\0') {
		const char* d = delim;
		while (*d != '\0') {
			if (*p == *d) {
				*p = '\0';
				*stringp = p + 1;
				return start;
			}
			d++;
		}
		p++;
	}

	*stringp = NULL;
	return start;
}

int strcoll(char* str1, char* str2) {
	int i = 0;
	while(str1[i] != '\0' && str2[i] != '\0') {
		if(str1[i] - str2[i] > 0)
			return 1;
		else if(str1[i] - str2[i] < 0)
			return -1;
		i++;
	}

	if(str1[i] == '\0' && str2[i] == '\0')
		return 0;
```

```
        else if(str1[i] == '\0')
                return -1;
        else
                return 1;
}

int main() {
        char s1[] = "Data", s2[] = "data", s3[] = "Structures", s4[] = "struct", s5[] = "data
structures and algorithms";
        char *p = s5, *token;

        printf("Comparison of strings using strcasecmp: %d\n", strcasecmp(s1, s2));
        printf("Tokenizing a string using strsep:\n");
        while ((token = strsep(&p, " ")) != NULL) {
                printf("%s\n", token);
        }
        printf("Comparison of strings using strcoll: %d\n", strcoll(s3, s4));

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q7.c -o Q7
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q7
Comparison of strings using strcasecmp: 0
Tokenizing a string using strsep:
data
structures
and
algorithms
Comparison of strings using strcoll: -1
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

8. What are wide characters? Write a sample code using wcscmp function. (You have to read wide character strings and then call wcscmp function)

Code:

```
#include <stdio.h>
#include <wchar.h>

int wcscmp(const wchar_t* s1, const wchar_t* s2) {
        while (*s1 && (*s1 == *s2)) {
```

```
		s1++;
		s2++;
	}
	return (int)(*s1 - *s2);
}

int main() {
	wchar_t str1[100], str2[100];

	wprintf(L"Enter first wide character string: ");
	fgetws(str1, 100, stdin);

	wprintf(L"Enter second wide character string: ");
	fgetws(str2, 100, stdin);

	str1[wcslen(str1) - 1] = L'\0';
	str2[wcslen(str2) - 1] = L'\0';

	int result = wcscmp(str1, str2);

	if (result == 0) {
		wprintf(L"The strings are equal.\n");
	} else if (result < 0) {
		wprintf(L"First string is less than second string.\n");
	} else {
		wprintf(L"First string is greater than second string.\n");
	}

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q8.c -o Q8
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q8
Enter first wide character string: wide
Enter second wide character string: wider
First string is less than second string.
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

9. Write following functions: sine, sine-inverse, cosine, cosine-inverse. Then using sine and cosine, write tan() function. Check whether calling sine(sine-inverse(x)) on your own functions gives you x.

Code:

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846
#define TERMS 10 // Number of terms in Taylor series

double factorial(int n) {
	double result = 1;
	for (int i = 1; i <= n; i++) {
		result *= i;
	}
	return result;
}

double sine(double x) {
	double result = 0;
	for (int i = 0; i < TERMS; i++) {
		int sign = (i % 2 == 0) ? 1 : -1;
		result += sign * pow(x, 2 * i + 1) / factorial(2 * i + 1);
	}
	return result;
}

double cosine(double x) {
	double result = 0;
	for (int i = 0; i < TERMS; i++) {
		int sign = (i % 2 == 0) ? 1 : -1;
		result += sign * pow(x, 2 * i) / factorial(2 * i);
	}
	return result;
}

double sine_inverse(double x) {
	double guess = x;
	for (int i = 0; i < TERMS; i++) {
```

```
		guess -= (sine(guess) - x) / cosine(guess); // Newton's method
	}
	return guess;
}

double cosine_inverse(double x) {
	return (PI / 2) - sine_inverse(x);
}

double tan(double x) {
	return sine(x) / cosine(x);
}

int main() {
	double x;

	printf("Enter a value in radians: ");
	scanf("%lf", &x);

	printf("Sine of %lf = %lf\n", x, sine(x));
	printf("Cosine of %lf = %lf\n", x, cosine(x));
	printf("Tangent of %lf = %lf\n", x, tan(x));

	printf("Enter the value of sine of some angle: ");
	scanf("%lf", &x);
	printf("Checking if sine(sine_inverse(%lf)) returns %lf\n", x, x);
	printf("sine(sine_inverse(%lf)) = %lf\n", x, sine(sine_inverse(x)));

	printf("Enter the value of cosine of some angle: ");
	scanf("%lf", &x);
	printf("Checking if cosine(cosine_inverse(%lf)) returns %lf\n", x, x);
	printf("cosine(cosine_inverse(%lf)) = %lf\n", x, cosine(cosine_inverse(x)));

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q9.c -o Q9 -lm
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q9
Enter a value in radians: 3.14158
Sine of 3.141580 = 0.000013
Cosine of 3.141580 = -1.000000
Tangent of 3.141580 = -0.000013
Enter the value of sine of some angle: 0.5
Checking if sine(sine_inverse(0.500000)) returns 0.500000
sine(sine_inverse(0.500000)) = 0.500000
Enter the value of cosine of some angle: 1
Checking if cosine(cosine_inverse(1.000000)) returns 1.000000
cosine(cosine_inverse(1.000000)) = 1.000000
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

10. Write a program to reverse the digits of an integer and store the result as another integer.

Code:

```
#include <stdio.h>

int main() {
	int num, rev = 0;
	printf("Enter a number: ");
	scanf("%d", &num);

	while(num) {
		rev *= 10;
		rev += (num % 10);
		num /= 10;
	}

	printf("Reversed number: %d\n", rev);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q10.c -o Q10
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q10
Enter a number: 19203
Reversed number: 30291
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

11. Write a program which reads a string and if the string has all digits in it, then derives the integer it represents.

Code:

```
#include <stdio.h>
#include <string.h>
#include <limits.h>

int str_to_int(char* str) {
	int flag = 0, num = 0, i;
	for(i = 0; i < strlen(str); i++) {
		char ch = str[i];
		if(ch >= 48 && ch <= 57) {
			num *= 10;
			num += ch - 48;
		}
		else {
			flag = 1;
			break;
		}
	}
	if(flag)
		return INT_MIN;
	return num;
}

int main() {
	char s[] = "165";

	printf("Given string: %s\n", s);
	int n = str_to_int(s);
	if(n == INT_MIN)
		printf("String does not contain only digits. It cannot be represented as an
integer.");
	else
		printf("Integer representation: %d\n", n);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q11.c -o Q11
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q11
Given string: 165
Integer representation: 165
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

12. Write a function which does the following void rev(char *str); Reverses the string "str" in place (without using another string).

Code:

```
#include <stdio.h>
#include <string.h>

void rev(char* str) {
	int i = 0, len = strlen(str);
	char temp;
	while(i <= len/2 - 1) {
		temp = str[i];
		str[i] = str[len - i - 1];
		str[len - i - 1] = temp;
		i++;
	}
	return;
}

int main() {
	char s[] = "data structures and algorithms";
	printf("Original string: %s\n", s);
	rev(s);
	printf("Reversed string: %s\n", s);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q12.c -o Q12
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q12
Original string: data structures and algorithms
Reversed string: smhtirogla dna serutcurts atad
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

14. Write a function which cuts a string given by "str" on the character given in "ch" and returns the first such word.
char *cutonchar(char *str, char ch);
For example: if 'str' is "something bad" and 'ch' is ' ' then it returns "something". if 'str' is "something bad" and 'ch' is 'e' then it returns 'som'

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* cutonchar(char* str, char ch) {
	int i = 0, n = strlen(str);
	char* new_str = (char*)malloc(n * sizeof(char)); //creating new string so that original string is not modified
	while(str[i] != ch && i < n) {
		new_str[i] = str[i];
		i++;
	}
	new_str[i] = '\0';
	return new_str;
}

int main() {
	char s[] = "DSA is an interesting course";

	printf("Original string: %s\n", s);
	printf("Modified string: %s\n", cutonchar(s, 'i'));

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ gcc -Wall Q14.c -o Q14
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$ ./Q14
Original string: DSA is an interesting course
Modified string: DSA
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Strings$
```

Structures

1. Declare a structure that represents the following hierarchical information:
(a) Student
(b) Roll Number
(c) Name (Typedef)
    (i) First name
    (ii) Middle Name
    (iii) Last Name
(d) Gender
(e) Date of Birth (Typedef)
    (i) Day
    (ii) Month
    (iii) Year
(f) Marks (typedef)
    (i) English
    (ii) Mathematics
    (iii) Computer Science

Code:

```
#include <stdio.h>
#include <string.h>

typedef struct name {
	char first_name[20];
	char middle_name[20];
	char last_name[20];
}Name;

typedef struct marks {
	float english;
	float mathematics;
	float comp_sci;
}Marks;

typedef struct date {
	int day;
	int month;
	int year;
```

```
}Date;

typedef struct student{
	int roll_no;
	Name name;
	char gender;
	Date dob;
	Marks marks;
}Student;

int main() {
	Student student1;

	student1.roll_no = 45;
	strcpy(student1.name.first_name, "Arnav");
	strcpy(student1.name.middle_name, "Rajat");
	strcpy(student1.name.last_name, "Singhal");
	student1.gender = 'M';
	student1.dob.day = 26;
	student1.dob.month = 7;
	student1.dob.year = 2004;
	student1.marks.english = 85.5;
	student1.marks.mathematics = 90.0;
	student1.marks.comp_sci = 92.0;

	printf("Roll number: %d\n", student1.roll_no);
	printf("Name: %s %s %s\n", student1.name.first_name,
student1.name.middle_name, student1.name.last_name);
	printf("Gender: %c\n", student1.gender);
	printf("Date of Birth: %02d/%02d/%d\n", student1.dob.day, student1.dob.month,
student1.dob.year);
	printf("Marks:\tEnglish: %.2f\tMathematics: %.2f\tComputer Science: %.2f\n",
student1.marks.english, student1.marks.mathematics, student1.marks.comp_sci);
	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q1.c -o Q1
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q1
Roll number: 45
Name: Arnav Rajat Singhal
Gender: M
Date of Birth: 26/07/2004
Marks:  English: 85.50  Mathematics: 90.00      Computer Science: 92.00
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

2. Using the above structure, write a program to display the details of the student whose name is entered by the user. Display the name of the students who have secured less than 40% of the aggregate. In addition, print each student's average marks.

Code:

```
#include <stdio.h>
#include <string.h>

typedef struct name {
	char first_name[20];
	char middle_name[20];
	char last_name[20];
}Name;

typedef struct marks {
	float english;
	float mathematics;
	float comp_sci;
}Marks;

typedef struct date {
	int day;
	int month;
	int year;
}Date;

typedef struct student{
	int roll_no;
	Name name;
	char gender;
	Date dob;
	Marks marks;
```

```
}Student;

int main() {
	Student students[] = {
	{12, {"Amit", "Ranjeet", "Agarwal"}, 'M', {25, 7, 2004}, {91, 77.5, 90}},
	{22, {"Bhumi", "Rajiv", "Bansode"}, 'F', {22, 4, 2004}, {57, 72, 80}},
	{35, {"Kirti", "Sachin", "Jadhav"}, 'F', {2, 10, 2004}, {95, 97, 90}},
	{45, {"Ninad", "Ashish", "Mehta"}, 'M', {15, 6, 2004}, {35, 47.5, 30}}
	};
	int i, flag = 0;
	char fname[20], mname[20], lname[20];

	printf("Enter the name to be searched: ");
	scanf("%s %s %s", fname, mname, lname);

	for(i = 0; i < 4; i++) {
		if(!strcmp(fname, students[i].name.first_name) && !strcmp(mname,
	students[i].name.middle_name) && !strcmp(lname, students[i].name.last_name))
	{
			flag = 1;
			printf("Name: %s %s %s\n", students[i].name.first_name,
		students[i].name.middle_name, students[i].name.last_name);
			printf("Roll No: %d\n", students[i].roll_no);
			printf("Gender: %c\n", students[i].gender);
			printf("Date of Birth: %02d/%02d/%d\n", students[i].dob.day,
		students[i].dob.month, students[i].dob.year);
			printf("Marks: English: %.2f | Mathematics: %.2f | Computer
		Science: %.2f\n\n", students[i].marks.english,
		students[i].marks.mathematics, students[i].marks.comp_sci);
			break;
		}
	}

	if(!flag)
		printf("Student not found\n\n");

	float aggregate;
	flag = 0;
	printf("Students who have secured less than 40%% of the aggregate:\n");
	for(i = 0; i < 4; i++) {
```

```
		if((aggregate = (students[i].marks.english + students[i].marks.mathematics
	+ students[i].marks.comp_sci) / 3.0) < 40) {
			flag = 1;
			printf("%s %s %s\t Aggregate = %.2f%%\n",
		students[i].name.first_name, students[i].name.middle_name,
		students[i].name.last_name, aggregate);
		}
	}
	if(!flag)
	printf("Nil\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q2.c -o Q2
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q2
Enter the name to be searched: Kirti Sachin Jadhav
Name: Kirti Sachin Jadhav
Roll No: 35
Gender: F
Date of Birth: 02/10/2004
Marks: English: 95.00 | Mathematics: 97.00 | Computer Science: 90.00

Students who have secured less than 40% of the aggregate:
Ninad Ashish Mehta       Aggregate = 37.50%
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

3. Write a program to define a structure for a hotel that has members— name, address, grade, number of rooms, and room charges. Write a function to print the names of hotels in a particular grade. Also write a function to print names of hotels that have room charges less than the specified value.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct hotel {
	char name[50];
	char address[100];
	int grade;
	int rooms;
	int charges;
```

```
}hotel;

void printHotelsByGrade(hotel hotels[], int num, int grade) {
	int i, flag = 0;
	for(i = 0; i < num; i++) {
		if(hotels[i].grade == grade) {
			flag = 1;
			printf("%s\n", hotels[i].name);
		}
	}
	if(!flag)
		printf("No hotels in this grade / Invalid grade\n");
	return;
}

void printHotelsByRoomCharges(hotel hotels[], int num, int charges) {
	int i, flag = 0;
	for(i = 0; i < num; i++) {
		if(hotels[i].charges < charges) {
			flag = 1;
			printf("%s\n", hotels[i].name);
		}
	}
	if(!flag)
		printf("No hotels with room charges below this limit\n");
	return;
}

int main() {
	int num_hotels = 3, grade, charges;

	hotel hotels[] = {
	{"Hotel A", "123 Main St, City A", 3, 50, 2000},
	{"Hotel B", "456 Oak St, City B", 4, 75, 8000},
	{"Hotel C", "789 Pine St, City C", 5, 60, 20000}
	};

	printf("Enter the required grade: ");
	scanf("%d", &grade);
	printHotelsByGrade(hotels, num_hotels, grade);
```

```
	printf("Enter the limit for room charges: ");
	scanf("%d", &charges);
	printHotelsByRoomCharges(hotels, num_hotels, charges);

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q3.c -o Q3
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q3
Enter the required grade: 4
Hotel B
Enter the limit for room charges: 20000
Hotel A
Hotel B
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

4. Declare a structure time that has three fields—hr, min, sec. Create two variables start_time and end_time. Input their values from the user. Then while start_time does not reach the end_time, display GOOD DAY on the screen.

Code:

```
#include <stdio.h>

typedef struct time {
	int hr;
	int min;
	int sec;
}time;

int cmp_time(time start_time, time end_time) { //return 0 if equal, 1 if start_time <
end_time and -1 otherwise
	if(start_time.hr < end_time.hr)
		return 1;
	else if(start_time.hr == end_time.hr) {
		if(start_time.min < end_time.min)
			return 1;
		else if(start_time.min == end_time.min) {
			if(start_time.sec < end_time.sec)
				return 1;
```

```
                else if(start_time.sec == end_time.sec)
                        return 0;
                return -1;
        }
        return -1;
    }
    return -1;
}

void increment_time(time *t) {
    t -> sec += 1;
    if(t -> sec == 60) {
        t -> sec = 0;
        t -> min += 1;
        if(t -> min == 60) {
            t -> min = 0;
            t -> hr += 1;
            if(t -> hr == 24) { //assuming 24 hour time format
                t -> hr = 0;
            }
        }
    }
    return;
}

int main() {
    time start_time, end_time;

    printf("Enter the start time: ");
    scanf("%2d:%2d:%2d", &start_time.hr, &start_time.min, &start_time.sec);

    printf("Enter the end time: ");
    scanf("%2d:%2d:%2d", &end_time.hr, &end_time.min, &end_time.sec);

    while(cmp_time(start_time, end_time) == 1) {
        printf("GOOD DAY\n");
        increment_time(&start_time);
    }

    return 0;
```

}

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q4.c -o Q4
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q4
Enter the start time: 08:50:05
Enter the end time: 08:50:10
GOOD DAY
GOOD DAY
GOOD DAY
GOOD DAY
GOOD DAY
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

5. Declare a structure fraction that has two fields— numerator and denominator. Create two variables and compare them using function. Return 0 if the two fractions are equal, –1 if the first fraction is less than the second and 1 otherwise. You may convert a fraction into a floating point number for your convenience.

Code:

```
#include <stdio.h>

typedef struct fraction {
	int numerator;
	int denominator;
}fraction;

int cmp_fraction(fraction f1, fraction f2) {
	int prod1, prod2;
	prod1 = f1.numerator * f2.denominator;
	prod2 = f2.numerator * f1.denominator;

	if(prod1 > prod2)
		return 1;
	else if(prod1 < prod2)
		return -1;
	else
		return 0;
}

int main() {
	fraction f1, f2;
```

```
	printf("Enter the first fraction: ");
	scanf("%d/%d", &f1.numerator, &f1.denominator);
	printf("Enter the second fraction: ");
	scanf("%d/%d", &f2.numerator, &f2.denominator);

	int cmp = cmp_fraction(f1, f2);

	if(cmp == 1)
		printf("%d/%d is greater than %d/%d\n", f1.numerator, f1.denominator,
f2.numerator, f2.denominator);
	else if(cmp == -1)
		printf("%d/%d is less than %d/%d\n", f1.numerator, f1.denominator,
f2.numerator, f2.denominator);
	else
		printf("The fractions are equal\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q5.c -o Q5
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q5
Enter the first fraction: 3/5
Enter the second fraction: 2/3
3/5 is less than 2/3
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

6. Define a structure date containing three integers— day, month, and year. Write a program using functions to read data, to validate the date entered by the user and then print the date on the screen. For example, if you enter 29/2/2010 then that is an invalid date as 2010 is not a leap year. Similarly 31/6/2007 is invalid as June does not have 31 days.

Code:

```
#include <stdio.h>

typedef struct Date{
	int day;
	int month;
	int year;
}Date;
```

```
int is_leap_year(int year) {
	if(year % 100 == 0) {
		if(year % 400 == 0)
			return 1;
		else
			return 0;
	}
	else if(year % 4 == 0)
		return 1;
	else
		return 0;
}

int is_valid_date(Date date) {
	if(date.month <= 7) {
		if(date.month == 2) {
			if(is_leap_year(date.year)) {
				if(date.day > 29)
				return 0;
			}
			else if(date.day > 28)
				return 0;
			}
			else {
			if(date.month % 2 && date.day > 31)
				return 0;
			else if(date.month % 2 == 0 && date.day > 30)
				return 0;
			}
		}
		else {
			if(date.month % 2 && date.day > 30)
				return 0;
			else if(date.month % 2 == 0 && date.day > 31)
				return 0;
	}

	return 1;
}
```

```
int main () {
	Date date;
	printf("Enter the day: ");
	scanf("%d", &date.day);
	printf("Enter the month: ");
	scanf("%d", &date.month);
	printf("Enter the year: ");
	scanf("%d", &date.year);

	printf("The date you entered is %02d/%02d/%d\n", date.day, date.month,
date.year);
	if(is_valid_date(date))
		printf("It is a valid date\n");
	else
		printf("It is an invalid date\n");

	return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q6.c -o Q6
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q6
Enter the day: 29
Enter the month: 02
Enter the year: 2004
The date you entered is 29/02/2004
It is a valid date
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

7. Write a program to add and subtract 10hrs 20mins 50sec and 5hrs 30min 40sec.

Code:

```
#include <stdio.h>

typedef struct time {
	int hours;
	int minutes;
	int seconds;
}time;

time get_struct(int hours, int minutes, int seconds) {
```

```
        time t;
        t.hours = hours;
        t.minutes = minutes;
        t.seconds = seconds;
        return t;
}

time add_time(time t1, time t2) {
        time t;
        int carry = 0;
        t.seconds = t1.seconds + t2.seconds;
        if(t.seconds >= 60) {
                carry = 1;
                t.seconds = t.seconds % 60;
        }
        t.minutes = t1.minutes + t2.minutes + carry;
        carry = 0;
        if(t.minutes >= 60) {
                carry = 1;
                t.minutes = t.minutes % 60;
        }
        t.hours = t1.hours + t2.hours + carry;
        return t;
}

time subtract_time(time t1, time t2) {
        time t;
        int borrow = 0;
        t.seconds = t1.seconds - t2.seconds;
        if(t.seconds <= 0) {
                borrow = 1;
                t.seconds = 60 + t.seconds;
        }
        t.minutes = t1.minutes - t2.minutes - borrow;
        borrow = 0;
        if(t.minutes <= 0) {
                borrow = 1;
                t.minutes = 60 + t.minutes;
        }
        t.hours = t1.hours - t2.hours - borrow;
```

```
        return t;
}

void display_time(time t) {
        printf("%dhrs %dmins %dsec\n", t.hours, t.minutes, t.seconds);
        return;
}

int main() {
        time t1, t2, sum, diff;
        t1 = get_struct(10, 20, 50);
        t2 = get_struct(5, 30, 40);
        sum = add_time(t1, t2);
        diff = subtract_time(t1, t2);
        display_time(sum);
        display_time(diff);
        return 0;
}
```

Output:

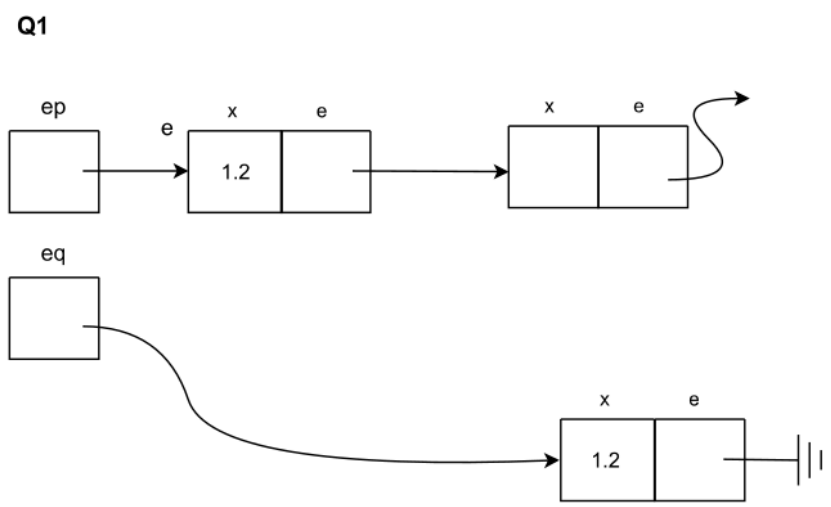
```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ gcc -Wall Q7.c -o Q7
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$ ./Q7
15hrs 51mins 30sec
4hrs 50mins 10sec
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Structures$
```

Pointers

Draw pointer diagrams for the following:

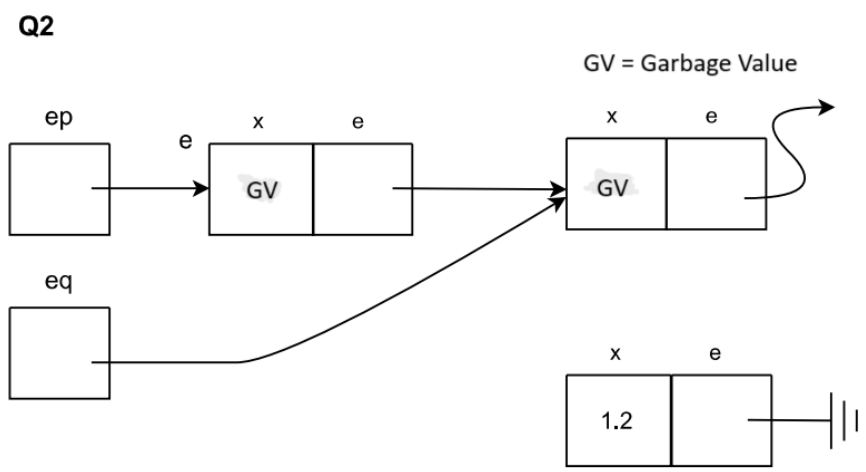
```
1) typedef struct entry {
        double x;
        struct entry *e;
}entry;
entry *f(entry **epp) {
        entry *ep = *epp;
        ep = (entry *)malloc(sizeof(entry));
        ep->e = NULL;
        ep->x = 1.2;
        return ep;
}
int main() {
        entry e, *ep, *eq;
        ep = &e; e.e = (entry *)malloc(sizeof(entry));
        eq = f(&(e.e));
        e.x = eq->x;
}
```

Solution:



```
2) typedef struct entry {
        double x;
        struct entry *e;
}entry;
entry *f(entry **epp) {
        entry *ep = *epp;
        *epp = (entry *)malloc(sizeof(entry));
        ep->e = NULL;
        ep->x = 1.2;
        return *epp;
}
int main() {
        entry e, *ep, *eq;
        ep = &e;
        e.e = (entry *)malloc(sizeof(entry));
        eq = f(&(e.e)); e.x = eq->x;
}
```

Solution:

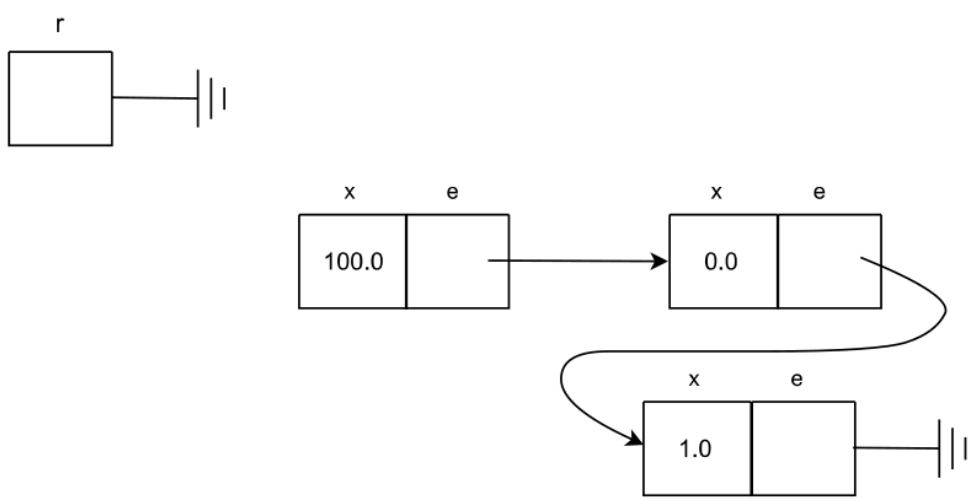



```
3) typedef struct entry {
        double x;
        struct entry *e;
}entry;
```

```
typedef entry *seq;
void f(seq *p) {
        int i;
        seq r;
        *p = (seq) malloc(sizeof(entry));
        r = *p; r->x = 100.0;
        for(i = 0; i < 2; i++) {
                r = (seq) malloc(sizeof(entry));
                (*p)->e = r;
                r->e = NULL;
                r->x = (double) i;
                (*p) = (*p)->e;
        }
}
int main() {
        entry *r;
        f(&r); r = r->e;
}
```

Solution:

Q3



Polynomial

Define ADT polynomial represented using an array. Write the following functions with suitable prototypes for ADT poly.
init_poly() // initializes polynomial
append() //adds a term in polynomial
display() //displays polynomial in proper format.
add_poly() // adds two polynomials and stores the result in third polynomial
sub_poly() // performs subtraction of two polynomials and stores the result in third polynomial
You are free to include more functions.

Skeleton of function main() is given below, use same by replacing commented statements by actual function calls:

```
int main() {
poly p;
//call init_poly()
// call append( )
// call display()
//call add_poly()
//call sub_poly()
return 0;
}
```

Code:

1) File - polynomial.h

```
typedef struct term{
	int coeff;
	int exp;
}term;

typedef struct poly{
	term* t;
	int size; //holds the number of terms that have been added
	int n;
}poly;

void init_poly(poly* p, int size);
void append(poly* p, int coeff, int exp);
```

```
void display(poly p);
void add_poly(poly* p1, poly* p2, poly* res);
void sub_poly(poly* p1, poly* p2, poly* res);
```

2) File - polynomial.c

```
#include <stdio.h>
#include <stdlib.h>

#include "polynomial.h"

void init_poly(poly* p, int n){
	p -> n = n;
	p -> size = 0;
	p -> t = (term*) malloc(sizeof(term) * n);
	return;
}

void append(poly* p, int coeff, int exp){
	p -> t[p -> size].coeff = coeff;
	p -> t[p -> size].exp = exp;
	p -> size++;
	return;
}

void display(poly p){
	int i, flag = 1;
	for (i = 0; i < p.size; i++){
		if(p.t[i].coeff == 0)
			continue;
		else {
			flag = 0;
			if(p.t[i].coeff > 0)
				printf(" %c %d", (i) ? '+' : ' ', p.t[i].coeff);
			else
				printf(" - %d", -p.t[i].coeff);
		}

		if(p.t[i].exp == 1)
			printf("x");
		else if(p.t[i].exp)
```

```
                        printf("x^%d", p.t[i].exp);
        }

        if(flag)
                printf("%d", 0);

        printf("\n");
        return;
}

void add_poly(poly* p1, poly* p2, poly* res) {
        for(int i = 0; i < p1->size; i++) {
                append(res, p1->t[i].coeff, p1->t[i].exp);
        }

        for(int i = 0; i < p2->size; i++) {
                int flag = 1;
                for(int j = 0; j < res->size; j++) {
                        if(res->t[j].exp == p2->t[i].exp) {
                                res->t[j].coeff += p2->t[i].coeff;
                                flag = 0;
                                break;
                        }
                }
                if(flag)
                        append(res, p2->t[i].coeff, p2->t[i].exp);
        }

        return;
}

void sub_poly(poly* p1, poly* p2, poly* res) {
        for(int i = 0; i < p1->size; i++) {
                append(res, p1->t[i].coeff, p1->t[i].exp);
        }

        for(int i = 0; i < p2->size; i++) {
                int flag = 1;
                for(int j = 0; j < res->size; j++) {
                        if(res->t[j].exp == p2->t[i].exp) {
```

```
                        res->t[j].coeff -= p2->t[i].coeff;
                        flag = 0;
                        break;
                }
            }
            if(flag)
                append(res, -(p2->t[i].coeff), p2->t[i].exp);
        }

        return;
}
```

3) File - main.c

```
#include "polynomial.c"

int main() {
        poly p1, p2, sum, diff;
        int n1, n2, i, coeff, exp;

        printf("Enter number of terms to be stored in the first polynomial: ");
        scanf("%d", &n1);
        printf("Enter number of terms to be stored in the second polynomial: ");
        scanf("%d", &n2);

        init_poly(&p1, n1);
        init_poly(&p2, n2);

        printf("Enter the coefficient and exponent of each term of the first polynomial:\n");
        for(i = 1; i <= p1.n; i++) {
                printf("Coefficient of term %d: ", i);
                scanf("%d", &coeff);
                printf("Exponent of term %d: ", i);
                scanf("%d", &exp);
                append(&p1, coeff, exp);
        }

        printf("Enter the coefficient and exponent of each term of the second
polynomial:\n");
        for(i = 1; i <= p2.n; i++) {
```

```
            printf("Coefficient of term %d: ", i);
            scanf("%d", &coeff);
            printf("Exponent of term %d: ", i);
            scanf("%d", &exp);
            append(&p2, coeff, exp);
        }

        printf("Polynomial 1: ");
        display(p1);
        printf("Polynomial 2: ");
        display(p2);

        init_poly(&sum, (n1 > n2) ? n1 : n2);
        add_poly(&p1, &p2, &sum);
        printf("Sum of the polynomials: ");
        display(sum);

        init_poly(&diff, (n1 > n2) ? n1 : n2);
        sub_poly(&p1, &p2, &diff);
        printf("Difference of the polynomials: ");
        display(diff);

        return 0;
}
```

Output:

```
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Polynomial$ gcc -Wall main.c -o main
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Polynomial$ ./main
Enter number of terms to be stored in the first polynomial: 3
Enter number of terms to be stored in the second polynomial: 4
Enter the coefficient and exponent of each term of the first polynomial:
Coefficient of term 1: 10
Exponent of term 1: 4
Coefficient of term 2: -5
Exponent of term 2: 2
Coefficient of term 3: 5
Exponent of term 3: 1
Enter the coefficient and exponent of each term of the second polynomial:
Coefficient of term 1: 9
Exponent of term 1: 4
Coefficient of term 2: 5
Exponent of term 2: 2
Coefficient of term 3: 2
Exponent of term 3: 1
Coefficient of term 4: 10
Exponent of term 4: 0
Polynomial 1:    10x^4 - 5x^2 + 5x
Polynomial 2:    9x^4 + 5x^2 + 2x + 10
Sum of the polynomials:    19x^4 + 7x + 10
Difference of the polynomials:    1x^4 - 10x^2 + 3x - 10
arnav@arnav-desktop-1:~/Desktop/Lab Work - 1/Polynomial$
```